

Resolución del Práctico 8

Testing de Software 2026

Tomás Rodeghiero

Índice

1. Ejercicio 1: NodeCachingLinkedList con Randoop	3
1.1. Flujo de trabajo	3
1.2. Defectos corregidos	3
1.3. Resultados	3
2. Ejercicio 2: fileExample con Randoop y EvoSuite	4
2.1. Randoop	4
2.2. EvoSuite	4
2.3. Comparación	5
3. Ejercicio 3: mecanismos de Randoop	5
3.1. Setup/teardown inyectado en tests	5
3.2. Fijar propiedades del sistema	5
3.3. Inyectar literales relevantes	5
3.4. Filtrar métodos bajo prueba	6
3.5. Clasificación de excepciones	6
3.6. Presupuesto y tamaño de la suite	6
3.7. Manejo de bloqueos	6
3.8. Captura de salida	6
3.9. Contratos extra con @CheckRep	6
3.10. Aplicación práctica	6
4. Ejercicio 4: IPBlacklist.login con EasyMock	6
4.1. Por qué usar mocks	7
4.2. Tests	7
5. Ejercicio 5: fail2ban.Server	9
5.1. (a) repOK()	9

5.2. (b) Randoop (30 s)	9
5.3. (c) EvoSuite (30 s)	10
5.4. (d) ¿repOK() para depurar con EvoSuite?	10
5.5. (e) Propiedad jqwik con un único generador	11
5.6. Resumen	12

1. Ejercicio 1: NodeCachingLinkedList con Randoop

Aplicar testing automático con **Randoop** sobre `assignment8_exercises.ncl.NodeCachingLinkedList`. A partir de la suite generada se analizan los defectos que aparecen, se corrigen en el código y se reportan las métricas de cobertura y mutación.

1.1. Flujo de trabajo

1. Compilar el proyecto y generar la suite inicial con Randoop.
2. Analizar las fallas detectadas para identificar defectos reales.
3. Corregir los defectos e incorporar el contrato `repOK()`.
4. Regenerar la suite, dejándola como tests de regresión.
5. Ejecutar JaCoCo (cobertura) y PIT (mutación).

1.2. Defectos corregidos

Randoop detectó dos problemas concretos en `NodeCachingLinkedList`:

1. **`maximumCacheSize` no se inicializaba en el constructor:** quedaba en 0 y deshabilitaba el cache.
2. **`repOK()` no estaba implementado:** devolvía `false` siempre.

Sobre esa base implementé `repOK()` completo y lo anoté con `@randoop.CheckRep` para que Randoop lo evalúe durante la generación.

1.3. Resultados

Suite de regresión final: **217 tests, 0 fallos, 0 errores.**

Métrica	Valor
Ramas cubiertas (JaCoCo)	25 / 70
Branch Coverage	35,71 %
Mutantes generados (PIT)	102
Mutantes muertos	39
Mutation score	38 %
Mutantes sin cobertura	44
Test strength	67 %

La suite generada por Randoop detecta una porción razonable de los mutantes que efectivamente llegan a ejecutarse (test strength 67 %), pero la baja cobertura de ramas limita el mutation score global.

2. Ejercicio 2: `fileExample` con Randoop y EvoSuite

`assignment8_exercises.fileContents.fileExample` expone una sola operación (`checkContent`) con varias dependencias fuertes con el entorno:

1. lee un nombre de archivo desde `System.in`,
2. cierra el `Scanner` de consola (lo que también cierra `System.in`),
3. si el archivo no existe, retorna `false`,
4. si existe, lee la primera línea,
5. la compara con la fecha actual formateada con `DateFormat.SHORT`.

2.1. Randoop

Salida: `RegressionTest0` con un único test que verifica la `NoSuchElementException` de `System.in` vacío. No se produjo `ErrorTest`.

Interpretación: Randoop explora muy poco de la clase porque su API pública exige IO real y estado externo para llegar a los caminos relevantes. Sin ese soporte, la generación aleatoria queda en el caso trivial.

2.2. EvoSuite

Salida:

- `fileExample_ESTest` (suite principal) con 3 tests.
- `fileExample_Failed_ESTest` con violaciones / errores detectados.

Métricas reportadas por EvoSuite durante la generación:

Métrica	Valor
Cobertura promedio	60 %
Line	53 %
Branch	60 %
Weak mutation	10 %

Al intentar ejecutar la suite localmente, falla al inicializar el runner de EvoSuite porque no encuentra `tools.jar`. El runtime está pensado para un JDK 8 con infraestructura histórica; sobre JDK 11+ no llega a inicializarse. Por eso la comparación se hace sobre las métricas reportadas por EvoSuite en la generación.

2.3. Comparación

	Randoop	EvoSuite
Velocidad	rápido	más lento
Casos generados	1 (trivial)	3 + suite Failed
ErrorTest	no	sí
Cobertura	baja	branch 60 % / line 53 %
Entorno requerido	JDK 17	JDK 11 (runtime histórico)

Ninguno reemplaza al otro: cada uno muestra un trade-off distinto entre profundidad de exploración y facilidad de integración.

3. Ejercicio 3: mecanismos de Randoop

En `fileExample.checkContent()` hay varias dependencias externas que dificultan la generación automática: entrada por `System.in`, lectura de archivos, dependencia de la fecha/locale, efecto lateral al cerrar el scanner de consola. Randoop ofrece opciones específicas para estas dificultades.

3.1. Setup/teardown inyectado en tests

`-junit-before-each`, `-junit-after-each`, `-junit-before-all`, `-junit-after-all`: permiten recrear `System.in`, preparar un archivo temporal controlado, y limpiar el estado al finalizar cada test.

3.2. Fijar propiedades del sistema

`-system-props` permite fijar `user.language`, `user.country`, `user.timezone` para estabilizar el formato de fecha.

3.3. Inyectar literales relevantes

`-literals-file` y `-literals-level` ayudan a que Randoop elija valores útiles (cadenas, rutas, identificadores).

3.4. Filtrar métodos bajo prueba

`-methodlist` y `-omitmethods` concentran la generación en las APIs relevantes y evitan ruido.

3.5. Clasificación de excepciones

`-checked-exception`, `-unchecked-exception`, `-npe-on-null-input`, `-npe-on-non-null-input`: cambian cómo se clasifica cada caso (`ErrorTest`, `RegressionTest`, o secuencia inválida).

3.6. Presupuesto y tamaño de la suite

`-timelimit`, `-inputlimit`, `-outputlimit`, `-maxsize`, `-small-tests`: priorizan tests legibles y evitan suites demasiado grandes.

3.7. Manejo de bloqueos

`-usethreads` y `-timeout` son claves cuando un camino puede quedar esperando entrada indefinidamente (como con `System.in`).

3.8. Captura de salida

`-capture-output` reduce el ruido cuando el código bajo prueba imprime por `stdout/stderr`.

3.9. Contratos extra con `@CheckRep`

Randoop evalúa el método anotado como invariante durante la generación y reporta violaciones. Especialmente útil en estructuras con estado interno.

3.10. Aplicación práctica

Para `fileExample`, la estrategia recomendada combina hooks de `setup/teardown` (`-junit-before-each`, `-junit-after-each`), fijación de propiedades (`-system-props`), acotamiento de suite (`-small-tests`, `-maxsize`, `-outputlimit`) y `timeout` por test para cortar bloqueos derivados de `stdin`.

4. Ejercicio 4: `IPBlacklist.login` con `EasyMock`

`IPBlacklist` tiene tres responsabilidades:

1. registrar el último IP que intentó loguearse,
2. contar los intentos fallidos consecutivos de ese IP,
3. agregar el IP a la lista negra cuando acumule 3 intentos fallidos consecutivos.

4.1. Por qué usar mocks

`IPBlacklist.login(...)` depende internamente de un `LoginService.login(ip, user, passwordHash)` que típicamente consulta un sistema externo. En los tests no queremos depender de ese servicio real: queremos controlar qué devuelve y verificar la lógica de blacklisting que está adentro de `IPBlacklist`. Con `EasyMock` se crea un doble del `LoginService` y se programa qué responder a cada llamada.

4.2. Tests

```
1 @Test
2 public void shouldBlacklistIpAfterThreeConsecutiveFailedAttempts
   () {
3     String ip = "192.168.0.10";
4     String user = "usuario";
5     String password = "clave";
6     String passwordHash = Utils.getPasswordHashMD5(password);
7
8     expect(service.login(ip, user, passwordHash)).andReturn(false
       ).times(3);
9     replay(service);
10
11     assertFalse(ipblacklist.login(ip, user, password));
12     assertFalse(ipblacklist.login(ip, user, password));
13     assertFalse(ipblacklist.login(ip, user, password));
14
15     assertTrue(ipblacklist.blacklisted(ip));
16     verify(service);
17 }
18
19 @Test
20 public void
   shouldNotBlacklistIpWhenFailedAttemptsAreLessThanThree() {
21     String ip = "192.168.0.20";
22     String user = "usuario";
23     String password = "clave";
24     String passwordHash = Utils.getPasswordHashMD5(password);
```

```
25
26     expect(service.login(ip, user, passwordHash)).andReturn(false
27         ).times(2);
28     replay(service);
29
30     assertFalse(ipblacklist.login(ip, user, password));
31     assertFalse(ipblacklist.login(ip, user, password));
32
33     assertFalse(ipblacklist.blacklisted(ip));
34     verify(service);
35 }
```

Esquema: `expect(...).andReturn(false).times(N)` programa el mock para devolver `false` en las N invocaciones siguientes; `replay(service)` pasa al modo “ejecución”; `verify(service)` confirma que el mock recibió exactamente las llamadas esperadas.

Resultado: los dos tests pasan.

5. Ejercicio 5: fail2ban.Server

Integra Randoop, EvoSuite y jqwik sobre `assignment8_exercises.fail2ban.Server`.

5.1. (a) repOK()

```
1 @CheckRep
2 public boolean repOK() {
3     if (expirationTime == null || expirationTime.longValue() <= 0
4         L) return false;
5     if (time == null) return false;
6     if (exceptions == null || bans == null) return false;
7     if (!exceptions.repOK() || !bans.repOK()) return false;
8
9     HashSet<IP> exceptionSet = exceptions.toSet();
10    HashSet<IP> banSet = bans.toSet();
11    for (IP ip : banSet) {
12        if (exceptionSet.contains(ip)) return false;
13    }
14
15    if (lastUpdate != null && !bans.greaterThan(lastUpdate))
16        return false;
17    return true;
18 }
```

Validaciones: `expirationTime` no nulo y > 0 ; `time` no nulo; `exceptions` y `bans` no nulos y consistentes con sus propios `repOK()`; ninguna IP en `exceptions` y `bans` a la vez; si `lastUpdate != null`, todos los bans con `expires > lastUpdate`.

5.2. (b) Randoop (30 s)

La suite inicial generada por Randoop era *flaky* por aserciones dependientes del tiempo del sistema. Regeneré con opciones estrictas (`-no-regression-assertions=true`, `-ignore-flaky-tests=true`, `-forbid-null=true`) y obtuve 174 tests en verde.

Defectos corregidos durante la depuración:

- `SinglyLinkedList.remove(...)`: eliminación correcta y decremento de `size`.
- Validaciones de `null` en operaciones públicas para evitar NPEs innecesarios.
- `StrictlySortedSinglyLinkedList.removeFromIP(...)` ahora devuelve `false` cuando no elimina nada.

- Mejoras de robustez en recorridos e inserciones sobre las listas.

Métricas (JaCoCo y PIT) sobre `Server`:

Métrica	Valor
Ramas cubiertas (JaCoCo)	2 / 52
Branch Coverage	3,85 %
Mutantes generados (PIT)	54
Mutantes muertos	2
Mutation score	4 %
Mutantes sin cobertura	50

5.3. (c) EvoSuite (30 s)

Métrica	Valor
Tests generados	30
Cobertura promedio	89 %
Branch coverage	89 % (48/54)
Weak mutation	92 % (65/71)
Mutation score	72 %

EvoSuite también generó `Server_Failed_ESTest` con escenarios que rompen el estado interno (por ejemplo, dejando `bans = null` desde un test dentro del mismo paquete) y disparan excepciones no declaradas.

Comparación Randoop vs EvoSuite

	Randoop	EvoSuite
Branch coverage	3,85 %	89 %
Mutation score	4 % (PIT)	72 % (propia)

EvoSuite ejercita la lógica de `Server` mucho más efectivamente. Su generación guiada por cobertura construye secuencias que llegan a ramas profundas, mientras que Randoop se queda en configuraciones superficiales.

5.4. (d) ¿`repOK()` para depurar con EvoSuite?

Sí, pero de manera distinta a Randoop:

- Randoop consume directamente la anotación `@CheckRep` y evalúa el invariante durante la generación.

- EvoSuite no la toma automáticamente. En su caso, `repOK()` se usa como oráculo explícito: o se agregan aserciones a los tests generados, o se envuelven las operaciones para invocarlo.

5.5. (e) Propiedad jqwik con un único generador

```
1 @Property(tries = 100)
2 void updateRemovesExactlyExpiredBans(@ForAll("updateScenarios")
   UpdateScenario scenario) {
3     Server server = new Server();
4     MutableTime time = new MutableTime(scenario.startTime);
5     server.setTime(time);
6
7     long[] expires = new long[scenario.ips.size()];
8     for (int i = 0; i < scenario.ips.size(); i++) {
9         IP ip = scenario.ips.get(i);
10        boolean added = server.addBan(ip);
11        Assertions.assertTrue(added);
12        expires[i] = time.getCurrentTime() + BAN_WINDOW_MS;
13        time.setNow(time.getCurrentTime() + 1L);
14    }
15
16    long nowAtUpdate = scenario.startTime + scenario.ips.size() +
        scenario.elapsed;
17    time.setNow(nowAtUpdate);
18    server.update();
19
20    for (int i = 0; i < scenario.ips.size(); i++) {
21        IP ip = scenario.ips.get(i);
22        boolean banStillActive = expires[i] > nowAtUpdate;
23        boolean expectedConnection = !banStillActive;
24        Assertions.assertEquals(expectedConnection, server.
            connect(ip));
25    }
26
27    Assertions.assertTrue(server.repOK());
28 }
```

El generador `updateScenarios()` produce un escenario completo (`UpdateScenario`) con una lista de IPs únicas, un tiempo inicial y un tiempo transcurrido. La propiedad verifica que tras `update()` cada IP queda correctamente permitida o bloqueada según su expiración, y que `repOK()` se sigue cumpliendo.

5.6. Resumen

- `repOK()` quedó implementado e integrado al flujo de testing automático.
- Randoop detectó problemas y permitió depurar, pero alcanzó baja cobertura y mutation score pobre.
- EvoSuite logró exploración de ramas y mutation score claramente superiores.
- La parte de PBT se completó con la propiedad `jqwik` para `update` usando un único generador.